

CSE312 HW2 REPORT

MUHAMMET FURKAN YILGIN
220104004964

DESIGN CHOICES

Directory Table and Directory Entries

The directory table contains entries for each file in the file system. Each directory entry includes:

- **File Name:** The name of the file (variable length).
- **Size:** The size of the file in bytes.
- **Owner Permissions:** Read (R) and Write (W) permissions.
- **Last Modification Date and Time:** The last time the file was modified.
- **File Creation Date and Time:** The time to state when the file is created.
- **Password:** The password protecting the file, if there is any.
- **First Block:** The index of the first block in the FAT chain for the file.

Free Blocks Management

Free blocks are managed using a bitmap (freeBlockBitmap). Each bit in the bitmap represents a block. If the bit is 0, the block is free; if the bit is 1, the block is occupied.

Handling Arbitrary Length File Names

File names are stored as C-style strings (null-terminated) in the directory entries. This allows for arbitrary length file names, constrained only by the available space in the directory entry.

Handling Permissions

Permissions are stored as a bitmask in the directory entry. The permissions bitmask is a 2-bit value:

- **Bit 0:** Read permission (R)
- **Bit 1:** Write permission (W)

For example:

- 0b00 = No permissions
- 0b01 = Read only
- 0b10 = Write only
- 0b11 = Read and Write

Handling Password Protection

Passwords are stored as C-style strings (null-terminated) in the directory entry. If a file is password-protected, the password must be provided to access the file.

Below you can see the structs and implementations of the functions for makeFileSystem file.

```

struct SuperBlock {
    uint32_t totalSize; // Total size of the file system
    uint32_t blockSize; // Size of each block in the file system
    uint32_t freeBlocksCount; // Number of free blocks in the file system
    uint32_t rootDirOffset; // Offset of the root directory in the file system
    uint32_t fatOffset; // Offset of the File Allocation Table (FAT) in the file system
    uint32_t dataOffset; // Offset of the data area in the file system
};

// Struct of directory entry in the file system
struct DirectoryEntry {
    char *fileName; // Name of the file or directory
    uint32_t size; // Size of the file in bytes
    uint16_t permissions; // Permissions of the file (e.g., read/write)
    time_t lastModification; // Last modification time of the file
    time_t creationTime; // Creation time of the file
    char *password; // Password for accessing the file (if protected)
    uint16_t firstBlock; // Index of the first data block allocated to the file
};

// Structure of an entry in the File Allocation Table (FAT) of the file system
struct fat_entry {
    uint16_t value; // Value indicating the next block in the chain (0xFFFF indicates end of chain)
};

// Structure of the file system
struct FileSystem {
    SuperBlock superBlock; // Super block of the file system
    std::vector<DirectoryEntry> directoryTable; // Vector storing directory entries
    std::vector<fat_entry> FAT; // Vector storing entries of the File Allocation Table (FAT)
    std::vector<bool> freeBlockBitmap; // Vector representing the free blocks in the file system
    std::vector<uint8_t*> dataBlocks; // Vector storing pointers to data blocks in the file system
};

```

- **createFileSystem(int blockSizeKB, const char *fileName):** Creates an empty file system.
- **formatFileSystem(FileSystem &fs):** Formats the file system
- **loadFileSystem(FileSystem &fs , const char *fileName) :** loads the file system.
- **saveFileSystem(FileSystem &fs, const char *fileName) :** saves the file system.

Implementations

```

// Functions
void createFileSystem(int blockSizeKB, const char *fileName); // Function to create the
file system
void formatFileSystem(FileSystem &fs); // Function to format the file system
void loadFileSystem(FileSystem &fs, const char *filename); // to load the file
void saveFileSystem(const FileSystem &fs, const char *filename); // to save the file

```

```

// Function to load the file system from a file
void loadFileSystem(FileSystem &fs, const char *filename) {
    std::ifstream in(filename, std::ios::binary);
    if (in.is_open()) {
        in.read(reinterpret_cast<char*>(&fs), sizeof(FileSystem));
        in.close();
    } else {
        std::cerr << "Could not load file system from " << filename << std::endl;
    }
}

```

```

void createFileSystem(int blockSizeKB, const char *fileName) {
    // Initializing the super block structure with appropriate values
    fs.superBlock.totalSize = MAX_FILE_SYSTEM_SIZE;
    fs.superBlock.blockSize = blockSizeKB * 1024;
    fs.superBlock.freeBlocksCount = (MAX_FILE_SYSTEM_SIZE / fs.superBlock.blockSize) - 1;
    fs.superBlock.rootDirOffset = sizeof(SuperBlock);
    fs.superBlock.fatOffset = fs.superBlock.rootDirOffset + sizeof(DirectoryEntry) * 128; // Assuming max 128 directory entries
    fs.superBlock.dataOffset = fs.superBlock.fatOffset + (fs.superBlock.totalSize / fs.superBlock.blockSize) * sizeof(fat_entry);

    // Creating a new file with the specified file name in binary mode
    std::ofstream fsFile(fileName, std::ios::binary);
    // Setting the file pointer to the end of the file
    fsFile.seekp(MAX_FILE_SYSTEM_SIZE - 1);
    // Writing a single byte to extend the file size to the maximum file system size
    fsFile.write("", 1);
    // Closing the file stream
    fsFile.close();
}

// Function to format the file system
void formatFileSystem(FileSystem &fs) {
    // Clearing the directory table
    fs.directoryTable.clear();
    // Initializing the File Allocation Table (FAT) with default values (0xFFFF indicates free block)
    fs.FAT.assign(fs.superBlock.totalSize / fs.superBlock.blockSize, {0xFFFF});
    // Initializing the free block bitmap (all blocks are initially free)
    fs.freeBlockBitmap.assign(fs.superBlock.totalSize / fs.superBlock.blockSize, false);
    // Updating the count of free blocks
    fs.superBlock.freeBlocksCount = fs.superBlock.totalSize / fs.superBlock.blockSize;

    // Initialize the data blocks vector
    fs.dataBlocks.resize(fs.superBlock.totalSize / fs.superBlock.blockSize);
    // Allocating memory for each data block
    for (auto &block : fs.dataBlocks) {
        // I used unsigned char for binary data
        block = new unsigned char[fs.superBlock.blockSize];
    }
}

```

- `./makeFileSystem makeFileSystem 1 mySystem.data ->` to run in the command line
- `./fileSystemOper fileSystem.data mkdir /ysa ->` to run in the command line
- Functions of file system operations are below with their implementations respectively.

FILE SYSTEM OPERATION FUNCTIONS(fileSystemOper)

```

void mkdir_fs(const std::string& path); // Function to create a director
void rmdir_fs(const std::string& path); // Function to remove a director
void duple2fs(); // Function to display file system information
void writeFile(const std::string& path, const std::string& fileName); // Function to
write data to a file -write
void readFile(const std::string& path, const std::string& fileName); // Function to
read data from a file -read-
void deleteFile(const std::string& path, const std::string& fileName); // Function
to delete a file
void chmod_fs(const std::string& path, const std::string& fileName, const
std::string& permissions); // Function to change file permissions
void addpw(const std::string& path, const std::string& fileName, const std::string&
password); // Function to add a password to a file
void dir(const std::string& path); // Function to list directory contents

```

DIR

```

// Function to list the contents of a directory
void dir(FileSystem &fs, const char *path) {
    // Traverse the directory table to find entries with the specified path
    for (const auto &entry : fs.directoryTable) {
        // Check if the entry's path matches the specified path
        if (strcmp(entry.fileName, path) == 0) {
            // Print directory contents
            for (const auto &dirEntry : fs.directoryTable) {

```

```

        // Check if the entry is in the specified directory
        if (strstr(dirEntry.fileName, path) == dirEntry.fileName) {
            // Print file details
            printf("File: %s\n", dirEntry.fileName);
            printf("Size: %u bytes\n", dirEntry.size);
            printf("Permissions: %c%c\n", (dirEntry.permissions & 0b01) ? 'R' :
'-' , (dirEntry.permissions & 0b10) ? 'W' : '-');
            printf("Last Modification: %s", ctime(&dirEntry.lastModification));
            printf("Creation Time: %s", ctime(&dirEntry.creationTime));
            printf("Password Protected: %s\n", (dirEntry.password ? "Yes" :
"No"));
        }
    }
    return;
}
// If directory not found, print error
printf("Directory not found: %s\n", path);
}

```

RMDIR

```

// Function to remove a directory
void rmdir_fs(FileSystem &fs, const char *path) {
    // Traverse the directory table to find entries with the specified path
    for (auto it = fs.directoryTable.begin(); it != fs.directoryTable.end(); ++it) {
        // Check if the entry's path matches the specified path
        if (strcmp(it->fileName, path) == 0) {
            // Erase the directory entry
            fs.directoryTable.erase(it);
            printf("Directory removed: %s\n", path);
            return;
        }
    }
    // If directory not found, print error
    printf("Directory not found: %s\n", path);
}

```

MKDIR

```

// Function to create a directory
void mkdir_fs(FileSystem &fs, const char *path) {
    // Create a new directory entry
    DirectoryEntry entry;
    entry.fileName = strdup(path);
    entry.size = 0;
    entry.permissions = 0b11; // Default permissions (read and write)
    entry.lastModification = time(NULL);
    entry.creationTime = time(NULL);
    entry.password = nullptr; // No password initially
    entry.firstBlock = 0xFFFF; // Indicates no blocks allocated

    // Add the new directory entry to the file system directory table
    fs.directoryTable.push_back(entry);
    printf("Directory created: %s\n", path);
}

```

DUMPE2FS

```
// Function to dump file system information
void dumper2fs(const FileSystem &fs) {
    // Print file system information
    printf("Total blocks: %u\n", fs.superBlock.totalSize / fs.superBlock.blockSize);
    printf("Block size: %u bytes\n", fs.superBlock.blockSize);
    printf("Free blocks: %u\n", fs.superBlock.freeBlocksCount);

    // Print occupied blocks and file names
    printf("Occupied blocks and files:\n");
    for (const auto &entry : fs.directoryTable) {
        uint16_t block = entry.firstBlock;
        printf("File: %s\n", entry.fileName);
        while (block != 0xFFFF) {
            printf("Block: %u\n", block);
            block = fs.FAT[block].value;
        }
    }
}
```

WRITE

```
void writeFile(FileSystem &fs, const char *path, const char *linuxFile) {
    // Check if the Linux file exists
    struct stat fileInfo;
    if (stat(linuxFile, &fileInfo) == -1) {
        perror("Could not access Linux file");
        return;
    }

    // Open the Linux file for reading
    int fd = open(linuxFile, O_RDONLY);
    if (fd == -1) {
        perror("Could not open Linux file");
        return;
    }

    // Allocate memory for file data
    char *data = (char *)malloc(fileInfo.st_size);
    // Read data from the Linux file
    read(fd, data, fileInfo.st_size);
    // Close the Linux file
    close(fd);
    // Extract file name from the path
    const char *fileName = strrchr(path, '/') + 1;
    // Create a new directory entry in the file system
    mkdir_fs(fs, fileName);
    // Write data to the file in the file system
    ::writeFile(fs, fileName, data); // Adjusted to match the function signature
    // Free allocated memory
    free(data);
    printf("File written: %s\n", path);
}
```

READ

```
// Function to read a file
void readFile(FileSystem &fs, const char *path, const char *linuxFile, const char
*password) {
    // Extract file name from the path
    const char *fileName = strrchr(path, '/') + 1;
    // Allocate memory for file data
    char *buffer = (char *)malloc(MAX_FILE_SYSTEM_SIZE); // Assuming MAX_FILE_SIZE is
defined elsewhere
    if (buffer == nullptr) {
        perror("Memory allocation failed");
        return;
    }
    // Traverse the directory table to find the specified file
    for (const auto &entry : fs.directoryTable) {
        // Check if the entry's file name matches the specified file name
        if (strcmp(entry.fileName, fileName) == 0) {
            // Check if the file is password protected and password is provided
            if (entry.password && strcmp(entry.password, password) != 0) {
                printf("Incorrect password.\n");
                free(buffer); // Free allocated memory before returning
                return;
            }

            // Read data from the file in the file system
            ::readFile(fs, fileName, buffer, password);
            // Open the Linux file for writing
            int fd = open(linuxFile, O_WRONLY | O_CREAT, entry.permissions);
            if (fd == -1) {
                perror("Could not open Linux file for writing");
                free(buffer); // Free allocated memory before returning
                return;
            }
            // Write data to the Linux file
            if (write(fd, buffer, entry.size) == -1) {
                perror("Error writing to Linux file");
                close(fd); // Close the file descriptor
                free(buffer); // Free allocated memory before returning
                return;
            }
            // Close the Linux file
            close(fd);
            free(buffer); // Free allocated memory before returning
            return;
        }
    }
    // If file not found, print error
    printf("File not found: %s\n", fileName);
    free(buffer); // Free allocated memory before returning
}
```

DEL

```
// Function to delete a file
void deleteFile(FileSystem &fs, const char *path) {
    // Extract file name from the path
    const char *fileName = strrchr(path, '/') + 1;
    // Traverse the directory table to find the specified file
    for (auto it = fs.directoryTable.begin(); it != fs.directoryTable.end(); ++it) {
        // Check if the entry's file name matches the specified file name
        if (strcmp(it->fileName, fileName) == 0) {
            // Erase the file entry
            fs.directoryTable.erase(it);
            printf("File deleted: %s\n", path);
            return;
        }
    }
    // If file not found, print error
    printf("File not found: %s\n", fileName);
}
```

CHMOD

```
// Function to change file permissions
void chmod_fs(FileSystem &fs, const char *path, uint16_t permissions) {
    // Extract file name from the path
    const char *fileName = strrchr(path, '/') + 1;
    // Traverse the directory table to find the specified file
    for (auto &entry : fs.directoryTable) {
        // Check if the entry's file name matches the specified file name
        if (strcmp(entry.fileName, fileName) == 0) {
            // Update file permissions
            entry.permissions = permissions;
            printf("Permissions changed for %s\n", path);
            return;
        }
    }
    // If file not found, print error
    printf("File not found: %s\n", fileName);
}
```

ADDPW

```
// Function to add a password to a file
void addpw(FileSystem &fs, const char *path, const char *password) {
    // Extract file name from the path
    const char *fileName = strrchr(path, '/') + 1;
    // Traverse the directory table to find the specified file
    for (auto &entry : fs.directoryTable) {
        // Check if the entry's file name matches the specified file name
        if (strcmp(entry.fileName, fileName) == 0) {
            // Allocate memory for the password
            entry.password = strdup(password);
            printf("Password added to %s\n", path);
            return;
        }
    }
}
```



```
    }  
}  
// If file not found, print error  
printf("File not found: %s\n", fileName);  
}
```